

```

# =====
# Name: Srushti Kale
# PRN: 23070521067
# Practical 11: Convolutional Autoencoder on MNIST Dataset
# =====

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist

# 1. Load and Preprocess MNIST Dataset
print("Loading MNIST Dataset...")
(x_train, _), (x_test, _) = mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

# Reshape images to (28, 28, 1) for Conv2D layers
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))

print(f"Training data shape: {x_train.shape}")
print(f"Testing data shape: {x_test.shape}")

# 2. Build Encoder-Decoder Architecture
input_img = Input(shape=(28, 28, 1))

# --- ENCODER ---
x = Conv2D(16, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(8, (3, 3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2, 2), padding='same')(x) # Compressed representation

# --- DECODER ---
x = Conv2D(8, (3, 3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2, 2))(x)
x = Conv2D(16, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

# 3. Compile the Autoencoder Model
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
autoencoder.summary()

# 4. Train Autoencoder
print("\n--- Training Autoencoder ---")
history = autoencoder.fit(
    x_train, x_train, # Note: input and target are both x_train
    epochs=10,
    batch_size=128,
    shuffle=True,
    validation_data=(x_test, x_test)
)

# 5. Compare Original and Reconstructed Images
print("\n--- Generating Reconstructions ---")
decoded_imgs = autoencoder.predict(x_test)

n = 10 # Number of digits to display

```

```
plt.figure(figsize=(20, 4))
for i in range(n):
    # Display Original Images
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == 0:
        ax.set_title("Original")

    # Display Reconstructed Images
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == 0:
        ax.set_title("Reconstructed")

plt.show()
```

Loading MNIST Dataset...

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 — 0s 0us/step

Training data shape: (60000, 28, 28, 1)

Testing data shape: (10000, 28, 28, 1)

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 28, 28, 16)	160
max_pooling2d (MaxPooling2D)	(None, 14, 14, 16)	0
conv2d_1 (Conv2D)	(None, 14, 14, 8)	1,160
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 8)	0
conv2d_2 (Conv2D)	(None, 7, 7, 8)	584
up_sampling2d (UpSampling2D)	(None, 14, 14, 8)	0
conv2d_3 (Conv2D)	(None, 14, 14, 16)	1,168
up_sampling2d_1 (UpSampling2D)	(None, 28, 28, 16)	0
conv2d_4 (Conv2D)	(None, 28, 28, 1)	145

Total params: 3,217 (12.57 KB)

Trainable params: 3,217 (12.57 KB)

Non-trainable params: 0 (0.00 B)

--- Training Autoencoder ---

Epoch 1/10

469/469 — 69s 142ms/step - loss: 0.1640 - val_loss: 0.0911

Epoch 2/10

469/469 — 65s 138ms/step - loss: 0.0878 - val_loss: 0.0840

Epoch 3/10

469/469 — 82s 137ms/step - loss: 0.0834 - val_loss: 0.0811

Epoch 4/10

469/469 — 65s 139ms/step - loss: 0.0810 - val_loss: 0.0791

Epoch 5/10

469/469 — 83s 142ms/step - loss: 0.0792 - val loss: 0.0775